FINAL PROJECT DRAFT

Automatic Firmware Generation for ML-Integrated IoT Wearables
A Modular Software Framework and Validation Case Study

Student Name: S.N.S.B.K.K. SEMASINGHE
Registration Number: IT22084668
Department / Faculty: Department of Software Engineering
Supervisor: Mrs. Hansi De Silva
Date: 2026-04-26

Declaration

I declare that this final project draft is prepared based on the provided research paper and that the content has been written in a structured academic format for project development purposes. The work presented here has been drafted to reflect the objectives, methodology, and validation approach described in the source paper. Any future final submission should be reviewed, refined, and aligned with institutional formatting requirements.

I further declare that the draft has been organized into the required sections, including background study, research gap, methodology, testing, and conclusions, to support completion of the final project report.

Signature: _____

Date : 2026-04-26

Abstract

IoT wearables often require repeated firmware scaffolding for sensors, data transport, configuration, and build workflows. This repetition slows development, increases maintenance effort, and makes research systems difficult to reproduce. The referenced paper proposes an automatic firmware generation framework for ESP32-based wearable devices, with a healthcare wristband used as a validation case study.

The generator is designed as a modular, configuration-driven software tool. It allows sensor selection, parameter settings, preview of generated files, and safe regeneration without overwriting user-owned code. The system uses a plugin-style sensor registry, template-based code generation, and ESP-IDF build integration to reduce manual coding effort. In the validation case study, the generated wristband firmware supports IMU, heart rate, and temperature sensing, with external machine learning services handling activity and cardiac analysis.

This final project draft presents the same concept in a structured academic format. It outlines the background literature, identifies the research gap and problem, defines the objectives, describes the methodology, and summarizes testing, implementation, and commercialization potential.

Acknowledgement

I would like to express sincere gratitude to my supervisor, lecturers, and academic department for their guidance and support throughout this project development process. Their feedback has helped shape the structure and direction of this draft.

I also acknowledge the authors of the referenced paper for providing a clear and practical foundation for this project topic. The ideas presented in the paper guided the organization of this draft, especially in the areas of firmware generation, modular software design, and wearable IoT validation.

Finally, I am grateful to my family and friends for their encouragement and patience during the preparation of this work.

Table of Contents

Declaration 2
Abstract 3
Acknowledgement 3
List of Tables 5
List of Figures 5
1. Introduction 6
1.1 Background literature 6
1.2 Research Gap 7
1.3 Research problem 7
1.4 Research Objectives 7
2. Methodology 8
2.1 System Architecture 8
2.2 Software Solution 8
2.2.1 Requirement Analysis 8
2.2.2 Feasibility Study 9
2.3 Initial Data Collection 9
2.4 Proposed System 9
2.5 Project Requirements 9
2.5.1 Functional Requirements 9
2.5.2 Non-functional Requirements 10
2.5.3 User Requirements 10
2.5.4 System Requirements 10
2.5.5 Software Requirements 10
2.5.6 Personnel Requirements 10
2.6 Commercialization 10
2.7 Testing and implementation 11
Unit Testing 11
Integration Testing 11
System Validation Testing 11
3. Result and discussion 12
4. Conclusion 12

Appendices 13
References 18

List of Tables

Table 1. Comparison of manual firmware development and automated firmware generation
Table 2. Functional requirements of the proposed system
Table 3. Non-functional requirements of the proposed system
Table 4. System requirements and software requirements
Table 5. Unit testing summary for the generator modules
Table 6. Integration testing summary for build and flash workflow
Table 7. Validation testing summary for the wearable use case

List of Figures

Figure 1. Overall system architecture of the firmware generator
Figure 2. Safe regeneration workflow and generated-code boundary
Figure 3. ML-integrated wearable validation workflow

1. Introduction

The rapid growth of IoT wearables has created a need for development tools that reduce repetitive firmware work. In many projects, developers repeatedly implement the same sensor drivers, communication logic, configuration handling, and build scripts. This slows prototyping and makes it harder to reproduce experimental results.

The referenced paper addresses this problem by proposing an automatic firmware generation framework for ESP32-based devices. The generator produces maintainable firmware from a structured configuration file and supports safe regeneration, modular sensor integration, and one-click build and flash operations.

For the final project, the same idea can be treated as a software engineering solution for accelerating embedded system development while keeping the generated output maintainable and reproducible.

1.1 Background literature

Previous IoT and embedded development workflows typically depend on manually written boilerplate code. Common tasks include initialization of sensors, sampling loops, data serialization, network transmission, and device-specific configuration. This repetitive effort increases development time and introduces inconsistencies across projects [1][2]. Although templates and scaffolding tools exist, many of them only partially address these challenges and do not fully support modular sensor selection or safe regeneration of code [3].

In wearable computing, development complexity is significantly higher because devices must manage multiple sensors concurrently, maintain responsive user interfaces, and communicate with external analytics services [4]. These constraints require efficient firmware architectures that balance performance, modularity, and maintainability.

When machine learning is integrated into IoT systems, firmware stability becomes critical while ML models continue to evolve independently. Many existing systems tightly couple firmware with cloud or ML services, making updates difficult and reducing system flexibility [5]. Therefore, separating firmware generation from ML pipelines has become an important design consideration in modern IoT research.

The following topics are relevant to the background literature of this project:

- IoT firmware boilerplate and repetitive development effort [1][2]
- Template-based code generation for embedded systems [3]
- Modular software design and plugin architectures [6]
- Reproducibility in research prototypes [7]
- Wearable health monitoring systems with external ML integration [4][5]

1.2 Research Gap

The paper shows that existing tools do not fully satisfy the needs of research-oriented IoT firmware development. Many frameworks provide only partial scaffolding, such as board templates or project skeletons, but they often leave sensor integration, configuration validation, and safe regeneration to the developer.

A clear gap exists between general-purpose project generators and a maintainable firmware generation system that can:

- select sensors through a structured configuration interface,
- preserve user edits during regeneration,
- validate hardware and pin constraints before generation, and
- support a reproducible workflow for wearable experiments.

This gap is especially visible in ML-enabled wearables, where firmware and model updates are frequently changed independently. A generator-first approach fills this gap by separating configuration, generation, and application logic.

1.3 Research problem

The core research problem is that developers spend excessive time writing and rewriting firmware boilerplate for sensor-based wearables. This creates several practical issues:

- development time increases because similar code is rewritten for every prototype,
- research results become harder to reproduce because each firmware version differs,
- sensor updates require refactoring across multiple files, and
- automatic regeneration can overwrite user modifications if the tool is not designed carefully.

Therefore, the problem is not only code generation itself, but the creation of a safe and maintainable generation workflow that can support rapid experimentation without breaking the firmware structure.

1.4 Research Objectives

The main objective of this project is to develop an automatic firmware generation framework that simplifies the creation of ESP32-based wearable applications.

- To generate firmware from a high-level configuration rather than manual coding.
- To support modular sensor selection and parameter binding.
- To preserve user-owned code during regeneration.
- To integrate build, flash, and monitoring operations into a simple workflow.
- To validate the framework using a healthcare wristband case study with ML integration.

2. Methodology

The methodology of the project follows a software engineering approach centered on requirement analysis, system design, implementation planning, and validation. The generator is treated as the main software product, while the healthcare wristband acts as the validation case study.

The overall development process is configuration-driven. A user selects board, transport, and sensors through the software interface, after which templates are rendered into a reproducible ESP-IDF project structure.

2.1 System Architecture

The proposed system contains five major layers:

- GUI layer for configuration, preview, and generation control
- Configuration model stored in a structured project file
- Sensor registry and validation layer
- Template rendering layer for code generation
- Build and flash integration layer for ESP-IDF

The architecture separates generated code from user-owned code. This separation makes regeneration safer because the system only overwrites files under the generated directory. The result is a maintainable workflow that supports iterative development.

Figure 1. Overall system architecture of the firmware generator

GUI configuration -> project configuration file -> template renderer -> generated project output -> build and flash workflow

The architecture is designed to reduce coupling between hardware selection, code generation, and deployment. It also allows the system to scale when new sensors are added through plugins.

2.2 Software Solution

The software solution is a desktop-based generator built around configuration management and template automation. The user does not manually write sensor boilerplate. Instead, the software creates the base firmware files, initializes selected modules, and prepares the ESP-IDF project structure.

This approach improves usability for researchers and developers who want to focus on system behavior rather than low-level code duplication.

2.2.1 Requirement Analysis

Requirement analysis focused on identifying the tasks that consume the most time in wearable firmware development. These include sensor setup, task scheduling, configuration handling, transport selection, and deployment. The analysis also considered the need to prevent accidental loss of user code during repeated generation.

From this analysis, the system was expected to support sensor selection, validation before generation, preview of file changes, and integration with standard ESP-IDF commands.

2.2.2 Feasibility Study

Technical feasibility is high because ESP-IDF, Python-based desktop tooling, and template engines are already mature technologies. Operational feasibility is also strong because the workflow aligns with how developers already build and flash ESP32 devices.

Economic feasibility is favorable since the approach reduces development time and can be implemented using open tools and reusable templates. Schedule feasibility is also acceptable because the system can be developed in modules and tested incrementally.

2.3 Initial Data Collection

Initial data collection was based on the selected paper, related wearable-IoT design concepts, and the requirements of an ESP32-based validation setup. The project also considers the sensor combinations described in the paper, including IMU, heart rate, and temperature sensing.

The collected information helps define the configuration schema, the expected file structure, and the validation logic for sensors, pins, and communication settings.

2.4 Proposed System

The proposed system is an automatic firmware generator for IoT wearables. The user chooses a board, transport method, and sensor set. The generator validates the configuration and then produces a structured ESP-IDF project with isolated generated files.

In the validation case study, the generated firmware supports an ESP32-based wristband that collects IMU, heart rate, and temperature data. The sensor data is sent to an external ML server, which returns activity and health-related feedback.

Figure 2. Safe regeneration workflow and generated-code boundary

User edits are preserved outside the generated directory. Only the auto-generated files are overwritten during regeneration, which reduces the risk of data loss and simplifies iteration.

2.5 Project Requirements

2.5.1 Functional Requirements

No.	Requirement
1	Allow the user to select board, transport, and sensors through a GUI.
2	Generate a complete ESP-IDF project from the selected configuration.
3	Validate pin assignments, sampling rates, and board capability before generation.
4	Provide a preview and diff of files before applying generation.
5	Preserve user-owned files outside the generated directory.
6	Support build, flash, and monitor actions from the software interface.

2.5.2 Non-functional Requirements

No.	Requirement
1	The generator should produce outputs consistently for the same configuration.
2	The regeneration process should not overwrite user modifications outside generated files.
3	The interface should be easy to use for non-expert developers.
4	The system should support reasonable generation speed for iterative development.
5	The software should remain maintainable and extensible through modular design.

2.5.3 User Requirements

Users need a simple workflow that reduces manual coding and makes firmware generation understandable. They should be able to configure hardware, preview output, and build firmware without deep knowledge of the internal project structure.

2.5.4 System Requirements

The system requires a compatible development environment for ESP-IDF, a Python runtime for the generator, and a supported host operating system for the GUI and build tools. A serial connection is needed for flashing and monitoring the ESP32 board.

2.5.5 Software Requirements

Component	Purpose
Python	Runs the generator application and supporting scripts.
PySide6	Provides the desktop GUI.
Jinja2	Renders project templates.
ESP-IDF	Builds, flashes, and monitors the firmware.
YAML	Stores configuration and sensor definitions.

2.5.6 Personnel Requirements

The project requires a software developer or embedded systems student, a supervisor for academic guidance, and optionally a tester familiar with ESP32 development and wearable device integration.

2.6 Commercialization

The project has commercial potential because many companies and startups build prototype wearable devices under tight timelines. A generator that reduces firmware development effort can save cost during prototyping and product experimentation.

Possible commercialization paths include:

- a desktop tool license for hardware and IoT development teams,
- a subscription-based template library for supported sensors and boards,
- a consulting or customization service for industrial clients, and
- an educational edition for universities and training labs.

The key value proposition is time saving, reproducibility, and easier maintenance of embedded firmware projects.

2.7 Testing and implementation

Testing and implementation are organized to verify that the generator works correctly, that the generated firmware builds successfully, and that the wearable validation case behaves as expected. Several testing methods are appropriate for this project.

Unit Testing

Unit testing focuses on individual generator modules such as configuration parsing, sensor validation, and template rendering. Each module is checked independently to ensure that errors are detected early.

Test Case	Input	Expected Output	Status
Configuration parsing	Valid YAML file	Parsed project configuration	Pass/Fail
Sensor validation	Conflicting pins	Validation error message	Pass/Fail
Template rendering	Selected sensors and board	Generated source files	Pass/Fail

Integration Testing

Integration testing checks how the GUI, configuration model, generator, and build tools work together. The objective is to confirm that the full workflow functions without breaking the project structure.

Test Case	Process	Expected Result	Status
Generate project	Select board and sensors, then generate	Valid project created	Pass/Fail
Build project	Run ESP-IDF build from the tool	Successful compilation	Pass/Fail
Flash firmware	Connect device and flash firmware	Firmware uploaded to board	Pass/Fail

System Validation Testing

System validation tests whether the generated wearable firmware can support the intended use case. In the reference paper, the validation application is a healthcare wristband that streams sensor data to an external ML server.

Test Case	Validation Focus	Expected Outcome	Status
IMU streaming	Data transmission rate and stability	Data sent at configured interval	Pass/Fail
Vitals collection	Heart rate and temperature capture	Accurate periodic reporting	Pass/Fail
ML feedback loop	Activity and health response handling	Correct endpoint interaction	Pass/Fail

These methods provide a clear path from module-level correctness to full system behavior. They also support the academic evaluation of the generator and the wearable validation case.

Figure 3 below represents the validation flow used in the case study.

Figure 3. ML-integrated wearable validation workflow

ESP32 wristband -> sensor data transmission -> external ML server -> activity and health feedback -> UI and fallback handling

3. Result and discussion

The expected result of the project is a working firmware generation framework that reduces boilerplate coding and improves maintainability for ESP32 wearable projects. The generator should produce consistent project structures and should support safe regeneration without removing user code.

The discussion from the source paper shows that the approach is particularly useful in ML-integrated wearables because the firmware can remain stable while the analytics pipeline evolves externally. This separation of concerns improves reproducibility and allows faster iteration during research.

From a software engineering perspective, the main strengths are modularity, configuration-driven generation, and a clear separation between generated and user-owned code. The main limitation is that support is initially centered on ESP32, so extension to other boards would require additional adapters and validation rules.

Overall, the project demonstrates that automatic firmware generation is not just a convenience feature. It is a practical approach for reducing embedded development overhead and for making research prototypes easier to reproduce and maintain.

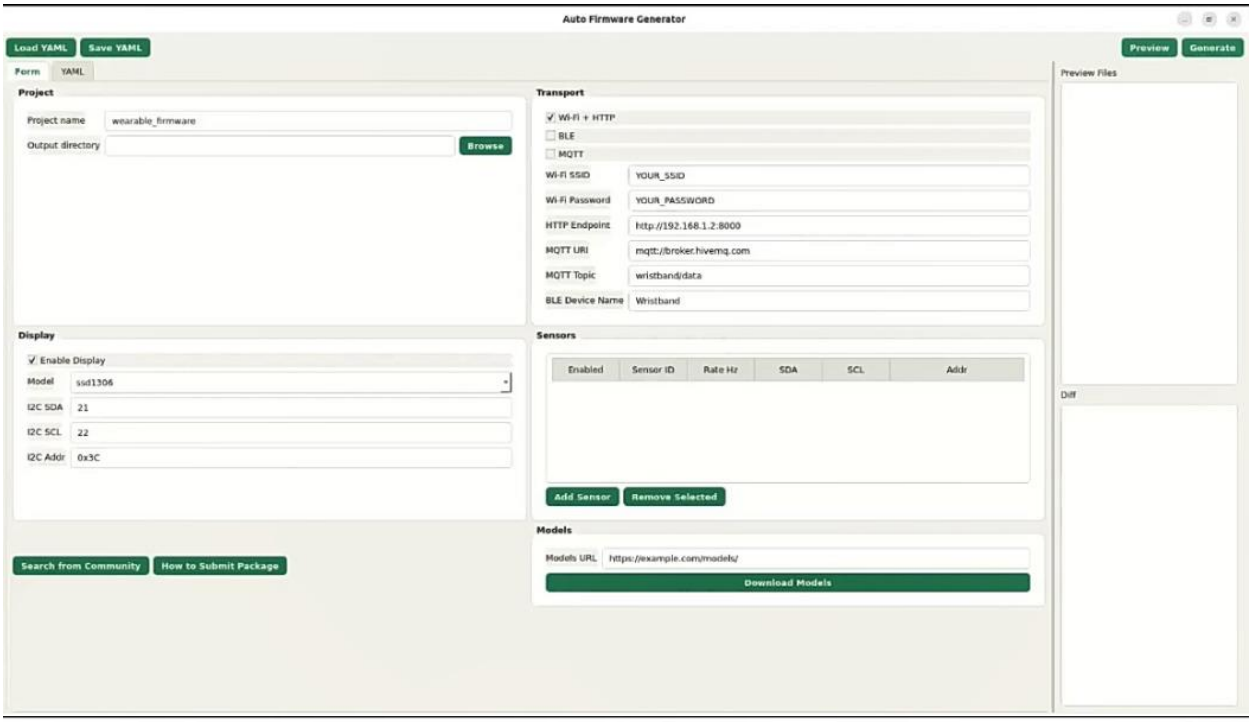
4. Conclusion

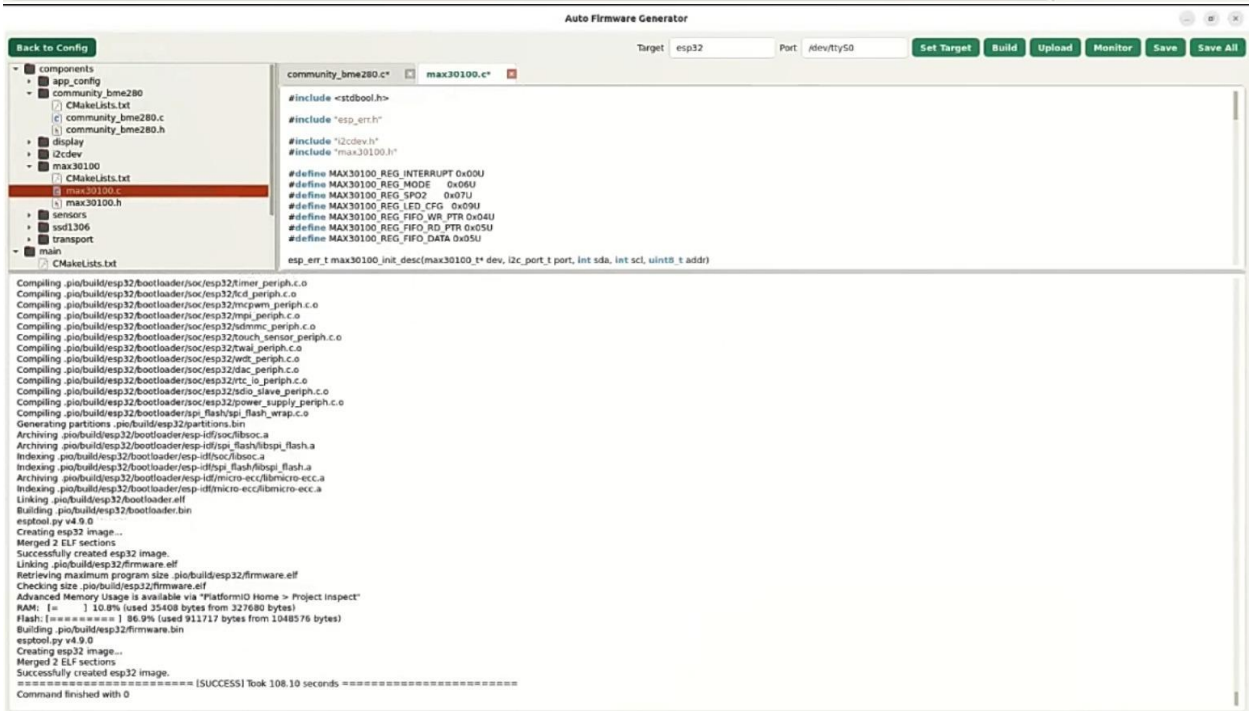
This final project draft presents an automatic firmware generation framework for ML-integrated IoT wearables, based on the provided research paper. The draft highlights the core idea that firmware development can be simplified by using a configuration-driven generator instead of manually rewriting repeated boilerplate.

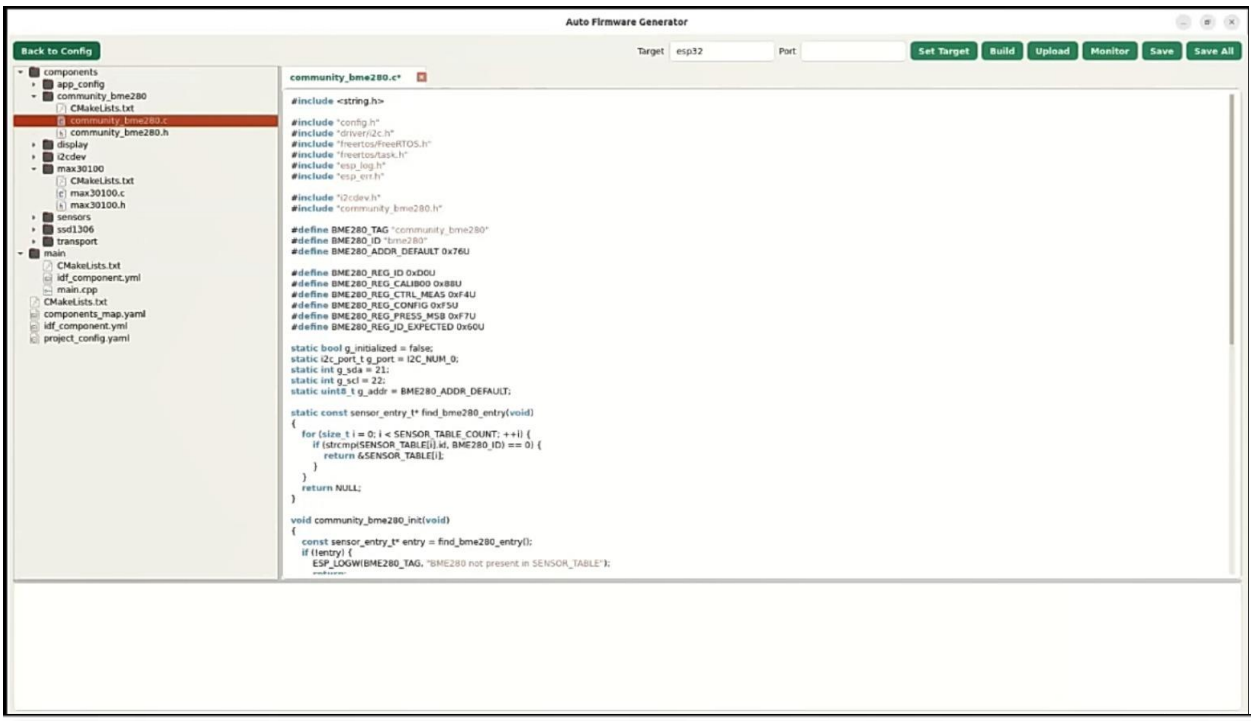
The methodology shows how the system can be organized into architecture, requirement analysis, feasibility study, proposed design, and testing. The validation case study demonstrates that the generated firmware can support a wearable healthcare application while preserving maintainability and reproducibility.

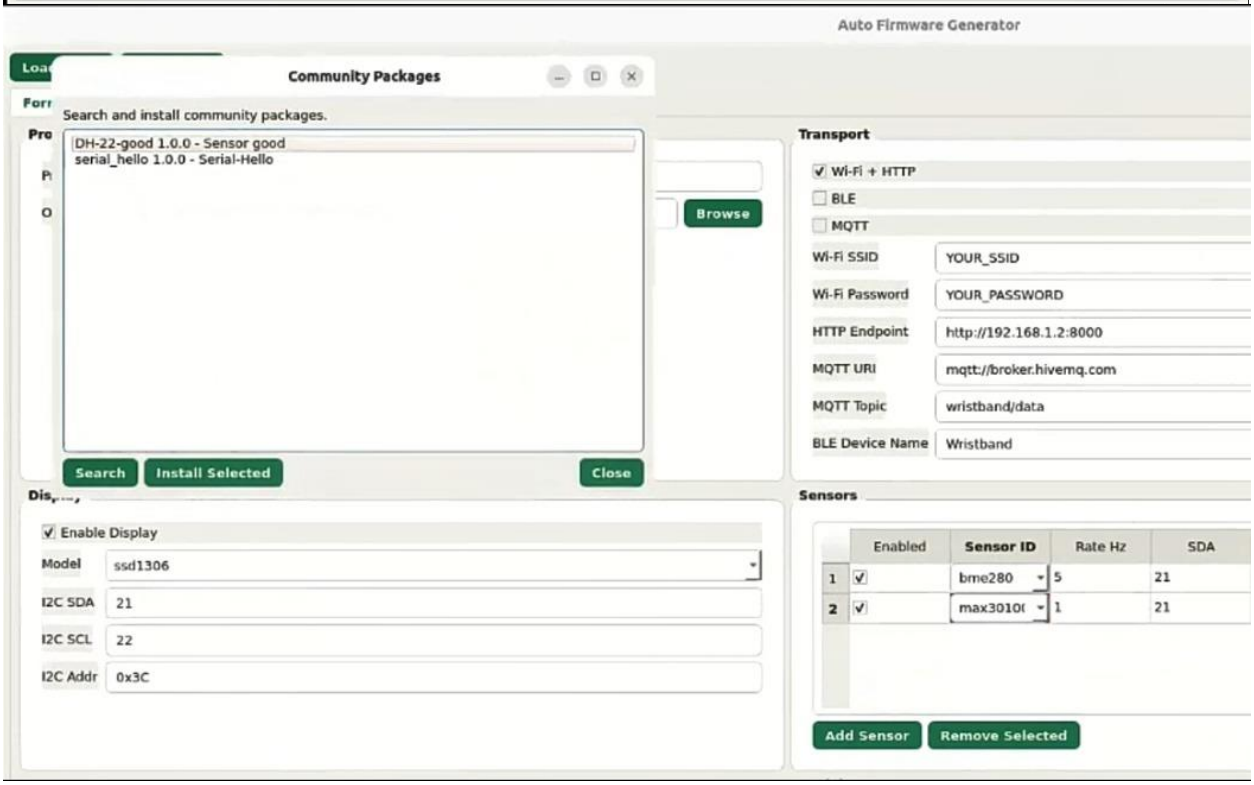
In conclusion, the project is suitable as a final-year software engineering topic because it combines embedded systems, software architecture, automation, and applied IoT validation in one coherent solution.

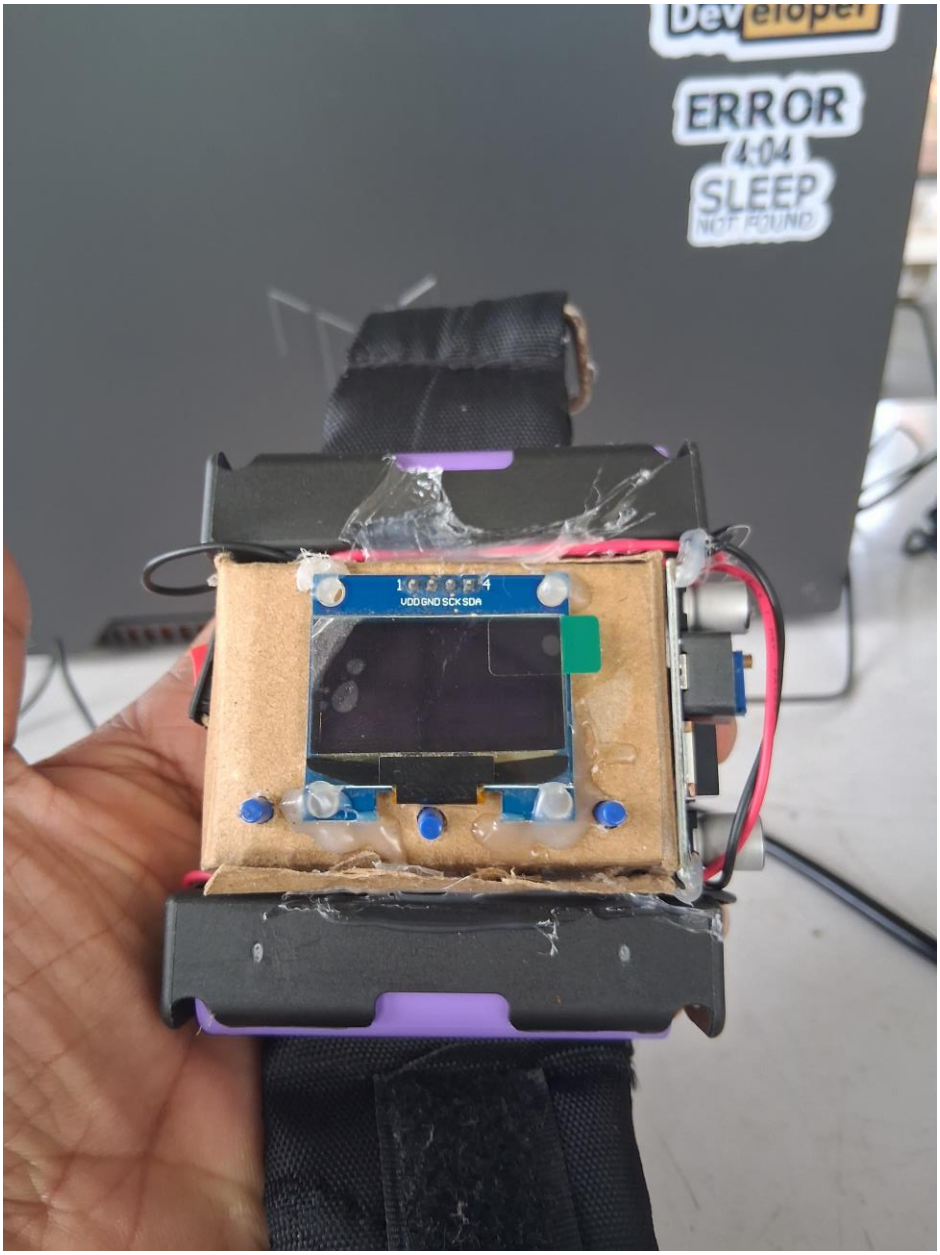
Appendices

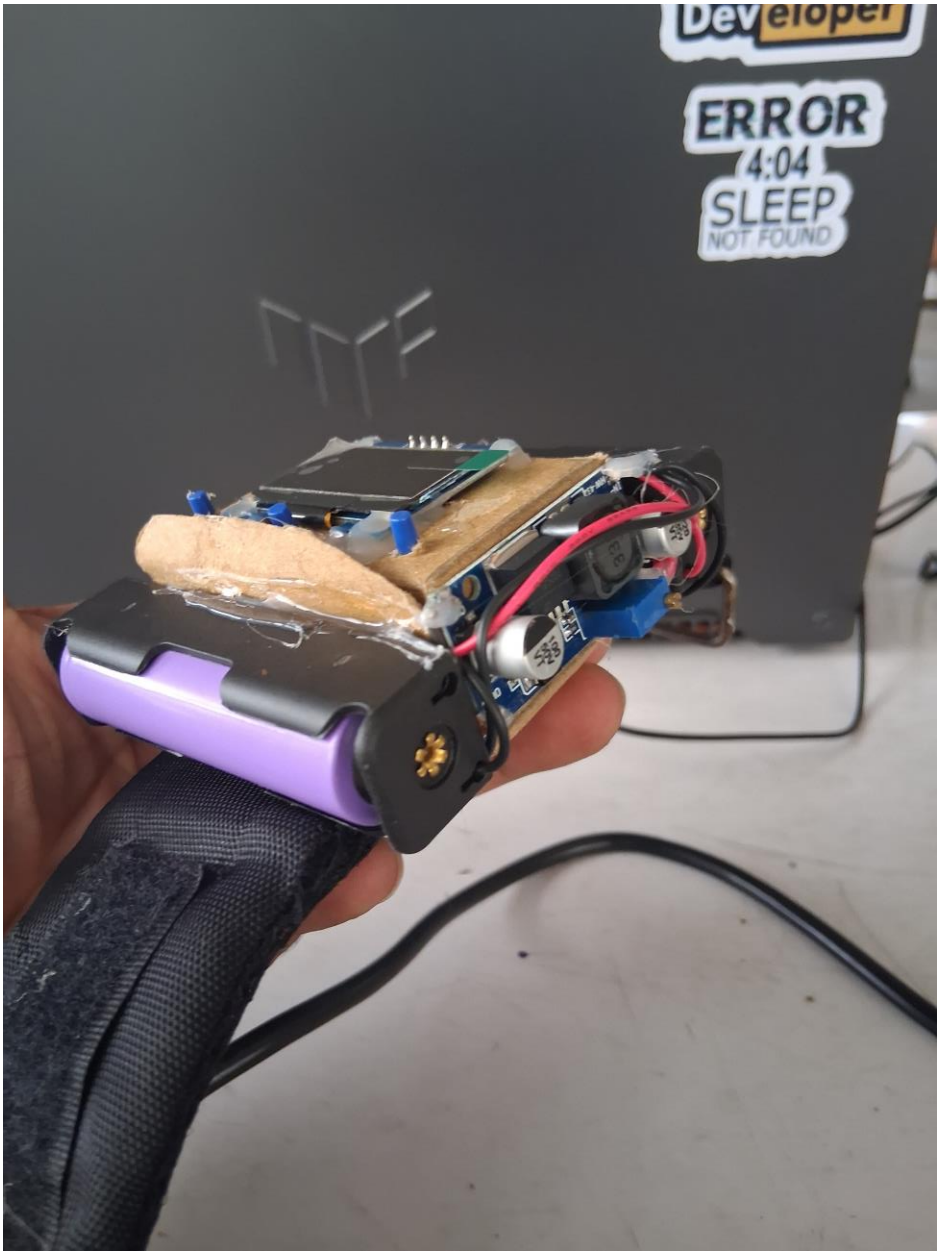

Auto Firmware Generator
Load YAML
Save YAML
Preview
Generate
Form
YAML
Project
Project name
wearable_firmware
Output directory
Browse
Transport
Wi-Fi + HTTP
BLE
MQTT
Wi-Fi SSID
YOUR_SSID
Wi-Fi Password
YOUR_PASSWORD
HTTP Endpoint
http://192.168.1.2:8000
MQTT URI
mqtt://broker.hivemq.com
MQTT Topic
wristband/data
BLE Device Name
Wristband
Preview Files
Display
Enable Display
Model
ssd1306
I2C SDA
21
I2C SCL
22
I2C Addr
0x3C
Sensors
Enabled
Sensor ID
Rate Hz
SDA
SCL
Addr
Add Sensor
Remove Selected
Diff
Search from Community
How to Submit Package
Models
Models URL
https://example.com/models/
Download Models

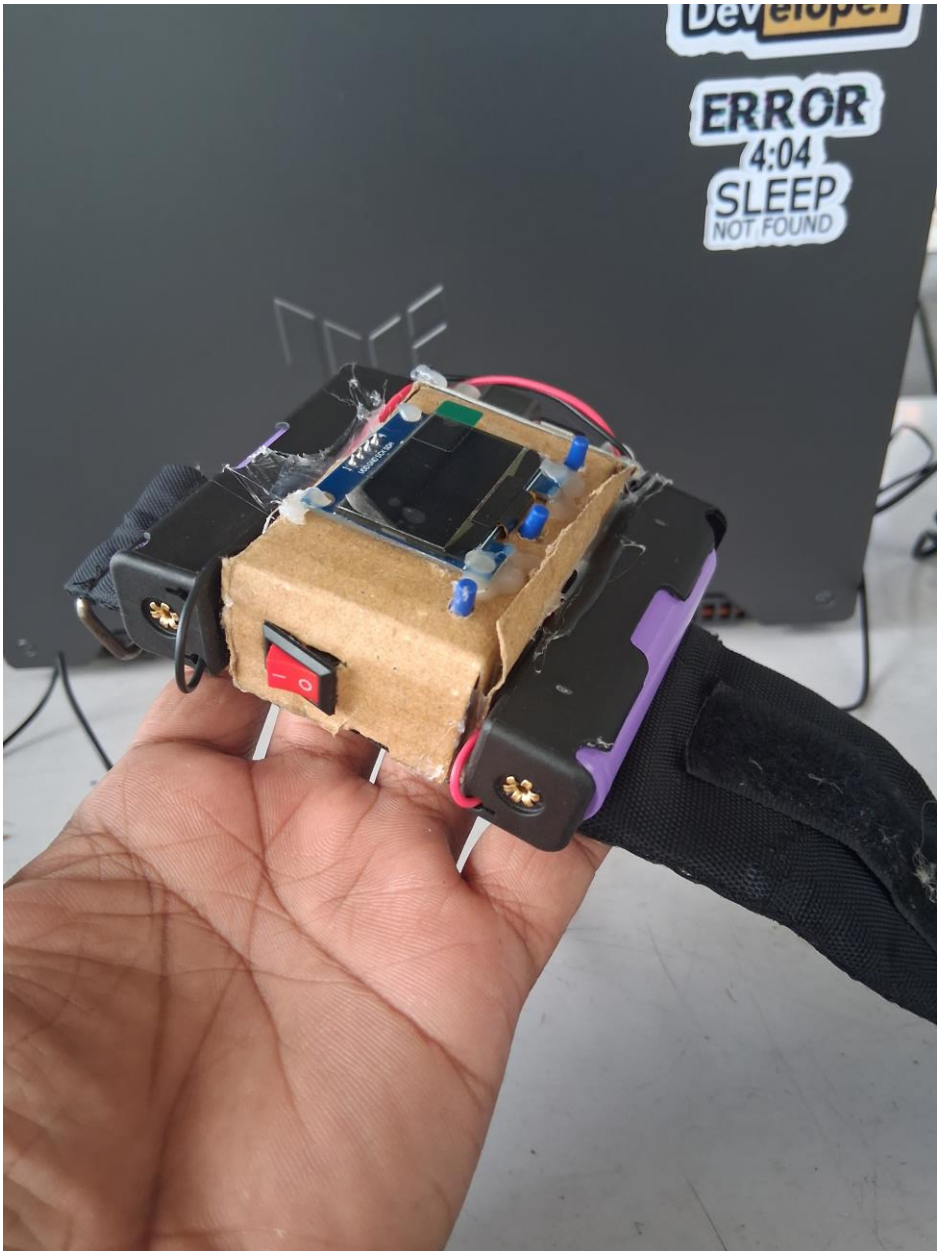

Auto Firmware Generator
Back to Config
Target
esp32
Port
/dev/ttyS0
Set Target
Build
Upload
Monitor
Save
Save All
components
app_config
community_bme280
CMakeLists.txt
community_bme280.c
community_bme280.h
display
i2cdev
max30100
CMakeLists.txt
max30100.c
max30100.h
sensors
ssd1306
transport
main
CMakeLists.txt
community_bme280.c*
max30100.c*
#include <stdbool.h>
#include "esp_err.h"
#include "i2cdev.h"
#include "max30100.h"
#define MAX30100_REG_INTERRUPT 0x00U
#define MAX30100_REG_MODE 0x06U
#define MAX30100_REG_SPO2 0x07U
#define MAX30100_REG_LED_CFG 0x09U
#define MAX30100_REG_FIFO_WR_PTR 0x04U
#define MAX30100_REG_FIFO_RD_PTR 0x05U
#define MAX30100_REG_FIFO_DATA 0x05U
esp_err_t max30100_init_desc(max30100_t* dev, i2c_port_t port, int sda, int scl, uint8_t addr)
Compiling .pio/build/esp32/bootloader/soc/esp32/timer_periph.c.o
Compiling .pio/build/esp32/bootloader/soc/esp32/lcd_periph.c.o
Compiling .pio/build/esp32/bootloader/soc/esp32/mcpwm_periph.c.o
Compiling .pio/build/esp32/bootloader/soc/esp32/mpi_periph.c.o
Compiling .pio/build/esp32/bootloader/soc/esp32/sdmmc_periph.c.o
Compiling .pio/build/esp32/bootloader/soc/esp32/touch_sensor_periph.c.o
Compiling .pio/build/esp32/bootloader/soc/esp32/twai_periph.c.o
Compiling .pio/build/esp32/bootloader/soc/esp32/wdt_periph.c.o
Compiling .pio/build/esp32/bootloader/soc/esp32/dac_periph.c.o
Compiling .pio/build/esp32/bootloader/soc/esp32/rtc_io_periph.c.o
Compiling .pio/build/esp32/bootloader/soc/esp32/sdio_slave_periph.c.o
Compiling .pio/build/esp32/bootloader/soc/esp32/power_supply_periph.c.o
Compiling .pio/build/esp32/bootloader/spi_flash/spi_flash_wrap.c.o
Generating partitions .pio/build/esp32/partitions.bin
Archiving .pio/build/esp32/bootloader/esp-idf/soc/libsoc.a
Archiving .pio/build/esp32/bootloader/esp-idf/spi_flash/libspi_flash.a
Indexing .pio/build/esp32/bootloader/esp-idf/soc/libsoc.a
Indexing .pio/build/esp32/bootloader/esp-idf/spi_flash/libspi_flash.a
Archiving .pio/build/esp32/bootloader/esp-idf/micro-ecc/libmicro-ecc.a
Indexing .pio/build/esp32/bootloader/esp-idf/micro-ecc/libmicro-ecc.a
Linking .pio/build/esp32/bootloader.elf
Building .pio/build/esp32/bootloader.bin
esptool.py v4.9.0
Creating esp32 image...
Merged 2 ELF sections
Successfully created esp32 image.
Linking .pio/build/esp32/firmware.elf
Retrieving maximum program size .pio/build/esp32/firmware.elf
Checking size .pio/build/esp32/firmware.elf
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
RAM: [=] 10.8% (used 35408 bytes from 327680 bytes)
Flash: [=========] 86.9% (used 911717 bytes from 1048576 bytes)
Building .pio/build/esp32/firmware.bin
esptool.py v4.9.0
Creating esp32 image...
Merged 2 ELF sections
Successfully created esp32 image.
========== [SUCCESS] Took 108.10 seconds ==========
Command finished with 0


Auto Firmware Generator
Back to Config
Target esp32
Port
Set Target
Build
Upload
Monitor
Save
Save All
components
app_config
community_bme280
CMakeLists.txt
community_bme280.c
community_bme280.h
display
i2cdev
max30100
CMakeLists.txt
max30100.c
max30100.h
sensors
ssd1306
transport
main
CMakeLists.txt
idf_component.yml
main.cpp
CMakeLists.txt
components_map.yaml
idf_component.yml
project_config.yaml
community_bme280.c*
#include <string.h>
#include "config.h"
#include "driver/i2c.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"
#include "esp_err.h"
#include "i2cdev.h"
#include "community_bme280.h"
#define BME280_TAG "community_bme280"
#define BME280_ID "bme280"
#define BME280_ADDR_DEFAULT 0x76U
#define BME280_REG_ID 0xD0U
#define BME280_REG_CALIB00 0x88U
#define BME280_REG_CTRL_MEAS 0xF4U
#define BME280_REG_CONFIG 0xF5U
#define BME280_REG_PRESS_MSB 0xF7U
#define BME280_REG_ID_EXPECTED 0x60U
static bool g_initialized = false;
static i2c_port_t g_port = I2C_NUM_0;
static int g_sda = 21;
static int g_scl = 22;
static uint8_t g_addr = BME280_ADDR_DEFAULT;
static const sensor_entry_t* find_bme280_entry(void)
{
for (size_t i = 0; i < SENSOR_TABLE_COUNT; ++i) {
if (strcmp(SENSOR_TABLE[i].id, BME280_ID) == 0) {
return &SENSOR_TABLE[i];
}
}
return NULL;
}
void community_bme280_init(void)
{
const sensor_entry_t* entry = find_bme280_entry();
if (!entry) {
ESP_LOGW(BME280_TAG, "BME280 not present in SENSOR_TABLE");


Auto Firmware Generator
Community Packages
Search and install community packages.
DH-22-good 1.0.0 - Sensor good
serial_hello 1.0.0 - Serial-Hello
Search
Install Selected
Close
Browse
Transport
Wi-Fi + HTTP
BLE
MQTT
Wi-Fi SSID
YOUR_SSID
Wi-Fi Password
YOUR_PASSWORD
HTTP Endpoint
http://192.168.1.2:8000
MQTT URI
mqtt://broker.hivemq.com
MQTT Topic
wristband/data
BLE Device Name
Wristband
Enable Display
Model
ssd1306
I2C SDA
21
I2C SCL
22
I2C Addr
0x3C
Sensors
Enabled
Sensor ID
Rate Hz
SDA
1
bme280
5
21
2
max3010(
1
21
Add Sensor
Remove Selected


Developer
ERROR
4:04
SLEEP
NOT FOUND
VDD GND SCK SDA


Developer
ERROR
4:04
SLEEP
NOT FOUND


ERROR
4:04
SLEEP
NOT FOUND

References

1. M. Broy, “Engineering Automotive Software,” *Proceedings of the IEEE*, 2006.
2. P. Clements et al., *Documenting Software Architectures*, Addison-Wesley, 2010.
3. E. Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, 1994.
4. A. Pantelopoulos and N. Bourbakis, “A Survey on Wearable Sensor-Based Systems,” *IEEE Transactions on Systems*, 2010.
5. L. Atzori et al., “The Internet of Things: A Survey,” *Computer Networks*, 2010.
6. D. Garlan and M. Shaw, “An Introduction to Software Architecture,” 1993.
7. J. Freire et al., “Reproducibility in Computational Science,” *Science*, 2012.